Programming Assignment 1: Preferential Web Crawler for MedOverNet

Tibor Koderman, Jerneja Krajcar, Dino Džaferagić

I. Introduction

In this project, we built a preferential crawler for the MedOverNet domain (https://medover.zurnal24.si/). The goal is to crawl relevant pages first, respect robots rules, avoid duplicate content, and store crawl data in PostgreSQL using the required crawldb schema.

The crawler is designed as a multi-worker system with a database-backed frontier. Each worker takes one URL, checks robots and politeness limits, downloads or renders the page, detects duplicates, stores the result, extracts new links and images, scores discovered URLs, and pushes them back to the frontier.

II. System architecture and implementation

A. High-level pipeline

The crawler processes each URL in the following order:

1) Claim next URL from the frontier.
2) On ingest, resolve canonical URL and reuse existing page records when available; at minimum, persist the `link` edge from source to target and avoid duplicate page rows.
3) Check robots rules and enforce politeness delay.
4) Download content (or render with headless browser when needed).
5) Detect exact duplicates using content hash.
6) Store page data and metadata in crawldb.
7) Parse outgoing links and images.
8) Score discovered URLs and enqueue them into frontier.

B. Main components

The implementation is split into clear modules:

- Worker orchestration and daemon control plane: `pa1/crawler/src/api/worker_service.py`, `reverse_channel.py`, `main.py`.
- Manager-side queue ownership and leases: `ManagerApp/Services/FrontierService.cs` and authenticated frontier endpoints.
- Crawler ingest and persistence integration: `ManagerApp/Services/CrawlerRelayService.cs`.
- Downloader and renderer: `pa1/crawler/src/core/downloader.py`, `renderer.py`.
- Parsing links and images: `pa1/crawler/src/core/parser.py`.
- Canonicalization, relevance, and robots/politeness: `url_canonicalizer.py`, `relevance.py`, `robots.py`.

C. Frontier queue semantics

The URL frontier is implemented in a dedicated table, `crawldb.frontier_queue`, rather than in `crawldb.page`. Each frontier row stores a canonical absolute `url`, `priority`, `source_url`, and `depth`, with a uniqueness constraint on `url`.

Enqueue uses UPSERT semantics and is concurrency-safe: if the URL already exists, priority is merged with `GREATEST`, depth/discovery metadata are kept from the best available values, and previously terminal rows (`COMPLETED`/`FAILED`) can be reactivated to `QUEUED`.

Workers claim URLs via an atomic transaction: candidates are read in priority order (`ORDER BY priority DESC, discovered_at ASC`) with `FOR UPDATE SKIP LOCKED`, then the selected row is moved to `LOCKED` and tagged with lease metadata (`locked_at`, `locked_by_worker_id`). Expired leases are automatically re-queued after a configurable TTL.

Queue state is enum-driven (`QUEUED`, `LOCKED`, `PROCESSING`, `COMPLETED`, `DUPLICATE`, `FAILED`). Completion reports map worker outcomes to terminal states, while retry outcomes map back to `QUEUED`. Politeness timing is enforced server-side through in-memory per-daemon/per-site cooldowns (minimum 5 s), not through a `frontier_next_fetch_time` column.

D. Division of work

We divided the implementation into three balanced workstreams and integrated all parts jointly in the final branch.

Tibor Koderman: architecture, database evolution, and manager platform

- base architecture integration and migration from direct crawler-DB coupling to server-owned queue flow,
- crawldb upgrade/migration work and operational scripts,
- deduplication integration in ingestion/persistence,
- multi-worker upgrade to manager-coordinated server mode,
- manager-side stats, analysis views, and visualization features.

Jerneja Krajcar: parsing/frontier and crawl-strategy stream

- parsing-dev branch work for link/image extraction and parser integration,
- frontier queue strategy material and related implementation refinements,
- preferential crawling and frontier documentation/validation updates,
- branch-level fixes and consolidation work merged into the final flow.

Dino Džaferagić: fetch/render and crawler-runtime stream

- downloader and renderer baseline implementation and testing,
- robots/rate-limiting foundations and runtime behavior improvements,
- crawler-side content-type handling and binary metadata handling groundwork,
- report drafting inputs, example collection, and final-text iteration support.

III. Preferential crawling strategy

The crawler uses a preferential strategy: every discovered URL is assigned a numeric priority (stored as `priority` in

`crawldb.frontier_queue`), and workers always claim the currently due URL with the highest priority.

A. Scoring function

The current implementation uses a lightweight, configurable heuristic (`relevance.py`). For a discovered URL `u` with parent URL `p` at depth `d`, we compute:

- **Same-host boost:** if `host(u) == host(p)`, add `same_host_boost` (default: 10).
- **Allowed domain suffix boost (optional):** if the host matches any configured suffix, add `allowed_suffix_boost` (default: 20).
- **Keyword boosts (optional):** for each configured keyword occurring in the URL path/query, add `keyword_boost` (default: 5).
- **Depth penalty:** subtract `depth_penalty` $\cdot d$ (default: 0.2).

With the default configuration, the crawler primarily prefers same-host URLs and mildly prefers shallow depth. Domain-specific focusing is supported via configuration, but must be explicitly set.

B. Worked example with default parameters

For a child URL discovered on the same host at depth $d = 3$:

$$\text{score} = 10 - 0.2 \cdot 3 = 9.4$$

For an off-host URL at the same depth (no same-host boost):

$$\text{score} = -0.6$$

Therefore, same-host pages are explored first unless additional boosts are configured.

IV. Robots.txt and politeness

The crawler enforces robots and ethics constraints before each fetch:

- robots fetched per origin and cached,
- `Allow`/`Disallow` checked for crawler user-agent,
- `Crawl-delay` read when present,
- `Sitemap` URLs collected for optional frontier seeding,
- hard minimum delay of 5 seconds per target IP, even if robots does not define delay.

The implementation uses a thread-safe per-IP limiter. Effective delay is:

$$\text{effective_delay} = \max(5s, \text{robots_crawl_delay})$$

This design allows parallel workers while still respecting per-target pacing.

V. Downloader, renderer, and content-type handling

A. HTTP downloader

The downloader uses `requests` with crawler user-agent and timeout settings from configuration. It follows redirects and stores status code and final URL.

B. JS rendering

For pages where static HTML is not enough, the crawler can render using headless Chrome via Selenium. This mode is optional because it is slower, and we use it when needed for dynamic content.

C. Content-type policy

The crawler distinguishes HTML and binary resources:

- HTML: stored in `page.html_content`, `page_type = HTML`
- PDF: supported as optional binary download, otherwise metadata-only
- DOC/DOCX/PPT/PPTX: tracked through `page_data` metadata (binary payload optional and configuration-gated)
- other non-HTML resources: marked as `BINARY`

In this assignment crawl HTML content by default, handle PDF where needed, and avoid storing unnecessary binary blobs.

VI. Deduplication

A. Exact deduplication

The crawler computes a content hash (SHA-256 of the exact HTML source) for HTML pages. The ingestion logic distinguishes:

- **Known canonical URL:** if the canonical URL already exists in `crawldb.page`, the crawler does not insert a new page row and does not treat this as a duplicate; it only records the hyperlink in `crawldb.link`.
- **Duplicate content:** if the canonical URL is new but the content hash matches an existing `HTML` page, a new row is inserted with `page_type_code = DUPLICATE`.

If the content hash already exists:

- new page is marked `DUPLICATE`,
- `duplicate_of` points to the original `HTML` page,
- duplicate page does not store full HTML content.

VII. Parameters and configuration

Crawler behavior is controlled by configuration variables. Main parameters include:

- number of workers,
- crawler user-agent (configured as `fri-wier-T`),
- minimum delay (must remain ≥ 5 s),
- robots timeout,
- download timeout,
- rendering timeout,
- optional PDF download flag.

The runtime behavior was validated with integration scripts and manager/crawler service runs:

- robots checks and effective delay behavior,
- downloader behavior for HTML/PDF cases,
- renderer output differences on dynamic pages,
- frontier claim/complete/requeue and ingest dedupe behavior.

VIII. Problems and solutions

A. Robots and ethical crawling

Problem: parallel workers can violate politeness if delays are not shared.
Solution: server-side politeness scheduling is applied per crawler/site identity before assigning work. Workers claim URLs through manager APIs, and the manager enforces minimum cooldown (and robots-derived delay when available) before a URL can be leased. This design allows parallel workers while preserving ethical pacing for target origins.

B. Dynamic content

Problem: some links/content are injected by JavaScript.
Solution: optional Selenium rendering path for selected pages.

C. Frontier concurrency

Problem: multiple workers may claim the same URL.
Solution: workers do not query PostgreSQL directly. They request leases from manager frontier endpoints, while the manager performs atomic claims in `crawldb.frontier_queue` using `FOR UPDATE SKIP LOCKED` and lease columns (`locked_at`, `locked_by_worker_id`). On completion/failure, manager updates terminal or retry states via lease token validation.

D. Duplicate handling

Problem: same content may appear under different URLs.
Solution: exact hash check and `DUPLICATE` page type with pointer to original.

E. Spider traps and loops

Problem: repeated navigation patterns and low-value loops can waste crawl budget.
Solution: canonical URL uniqueness in DB, relevance scoring, and robots compliance constraints.

F. Binary type accounting bug and correction

Problem: during validation we identified an unnoticed accounting bug in per-type binary statistics. The original dashboard/report logic relied on `page_data.data_type_code` and strict file-suffix checks on the URL path only. In practice, many collected binary resources had no `page_data` row (metadata-only ingest), while some type hints appeared in query parameters or response headers rather than in the path. As a result, part of the binary corpus was undercounted in type breakdowns.

Solution: we corrected the pipeline in two steps:

- online counting now classifies all `BINARY` rows from `crawldb.page` (not only `page_data`),
- type detection now uses content-type plus filename hints from URL path, query parameters, and `Content-Disposition` when available.

We also added a separate image-type breakdown (JPG/PNG/WEBP/GIF/SVG/BMP/TIFF/ICO/AVIF/OTHER), which matches the current crawl profile much better.

Recovery without full recrawl: yes, this can be fixed without crawling from scratch. Existing `BINARY` URLs already stored in `crawldb.page` can be reclassified offline with the improved rules (plus optional targeted `HEAD` checks only for unresolved `OTHER` entries). A full crawl restart is not required for corrected aggregate statistics.

IX. Results, statistics, and visualization

This section summarizes the final crawl snapshot and includes screenshots from the manager UI. All numeric values below were extracted from the project PostgreSQL database (`crawldb`) after the run.

A. Crawldb statistics

The seed-domain subset below refers to `medover.zurnal24.si`.

Table I
Main crawl statistics (seed domain and full crawldb)

Metric	Seed domain (MedOverNet)	Full crawldb
Number of sites	1	280
Number of HTML pages	36	1001
Number of duplicates	0	27
Number of binary pages	583	16786
Number of images	807	37210
Average images per HTML page	22.42	37.17

Table II
Binary documents by type

Data type	Count
PDF	19
DOC	0
DOCX	0
PPT	0
PPTX	0

In this specific snapshot, document counters are low (PDF=19, other office types currently 0), while the binary corpus remains heavily image/media dominated.

B. Manager dashboard overview

Figure 1 shows the aggregate crawl status and live worker/frontier monitoring.

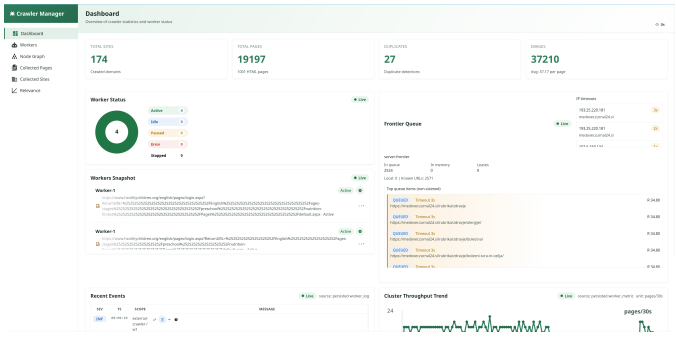

Figure 1. Crawler manager dashboard with final aggregate counters and live frontier status.

C. Worker runtime snapshot

Figure 2 shows active daemon workers, per-worker processed/error counts, and current worker states.

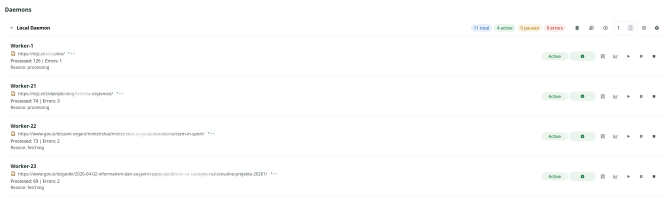

Figure 2. Workers view during crawl execution (active workers with per-worker progress).

D. Collected pages inspection

Figure 3 shows collected page entries and metadata (HTTP status, robots info, score, backlinks, extracted images).

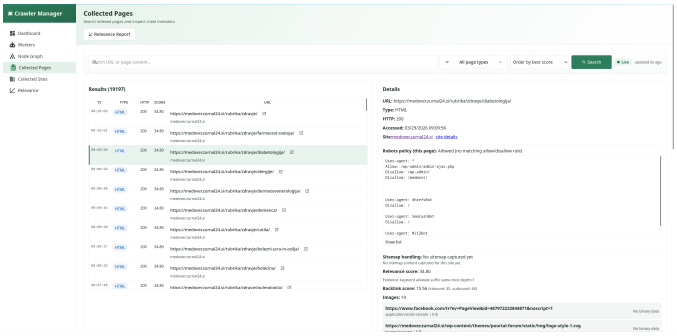

Figure 3. Collected pages view used to inspect crawl metadata and indexed content.

E. Relevance results

We calculated the final relevance report by evaluating the top 5000 scored pages in the database. This estimate is not a random-sample measurement and depends on crawl budget/runtime, but it provides a practical signal of how well preferential ordering surfaces domain-relevant pages. For the Assignment 1 relevance target sample, the manager report shows:

- top scored sample / target: 5000 / 5000,
- estimated relevant: 1668 (33.4%),
- keyword evidence: 1122,
- average / median / top score: 6.42 / 0.10 / 34.80.

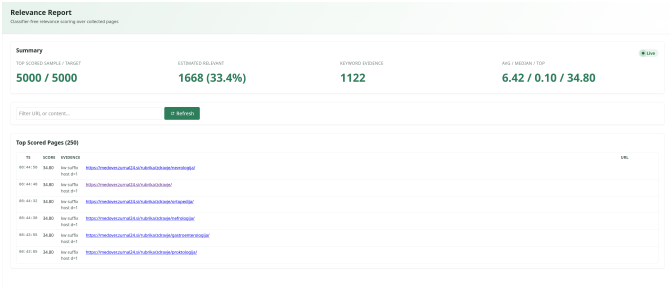

Figure 4. Relevance report summary and top-scored URLs over the 5000-page target sample.

F. Link graph visualization

The cross-domain link graph is shown in Figure 5.

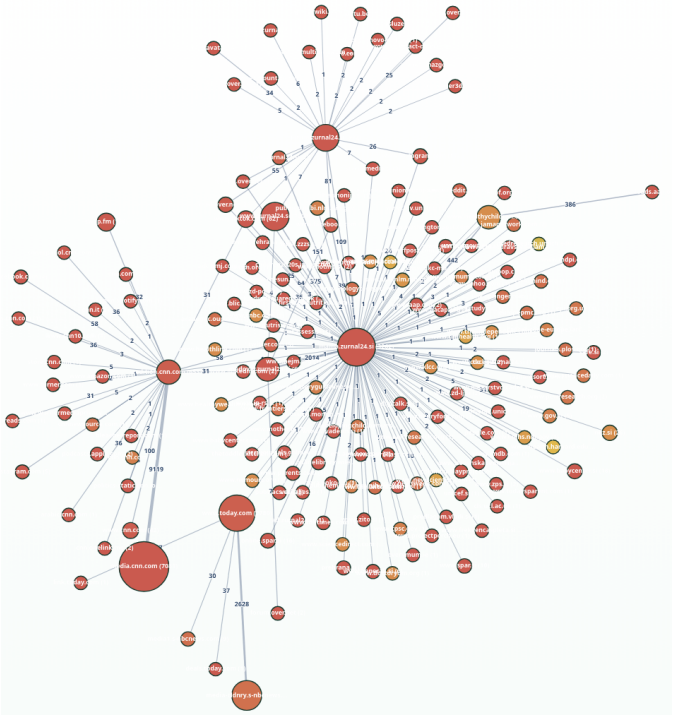

Figure 5. Crawl link graph (nodes = sites/pages, edges = discovered links).

X. Conclusion

We implemented and validated a preferential crawler pipeline with robots compliance, politeness enforcement, deduplication, queue-based worker coordination, and database persistence. The final crawl snapshot contains 1001 HTML pages, 27 exact duplicates, and 37210 images across 280 sites, while the relevance report reached the 5000/5000 target sample with 1668 estimated relevant pages (33.4%).